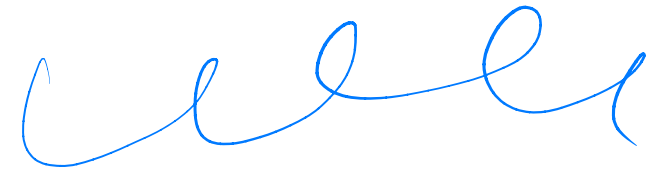


EECS 498/CSE 598: Zero-Knowledge Proofs

Winter 2026

Lecture 17



Paul Grubbs

paulgrub@umich.edu

Beyster 4709

Agenda for this lecture

- Announcements
- Proving memory accesses in CPU-model frontends
- Merkle trees
- memory fingerprinting
- Lookup arguments: doing better for read-only

Agenda for this lecture

- **Announcements**
- Proving memory accesses in CPU-model frontends
- Merkle trees
- memory fingerprinting
- Lookup arguments: doing better for read-only

Announcements

- Project 3 online, **due 3/24**
- Theory track problem set 1 online, **due 3/24**
- The plan is to release P4 and theory pset 2 tomorrow.
 - Note there will be no P5 nor a final exam. These will be the last assignments of the semester.
- 598 research track: **project proposal due Friday 3/27**
- All work for this course **must** be completed by our final exam date, April 27th

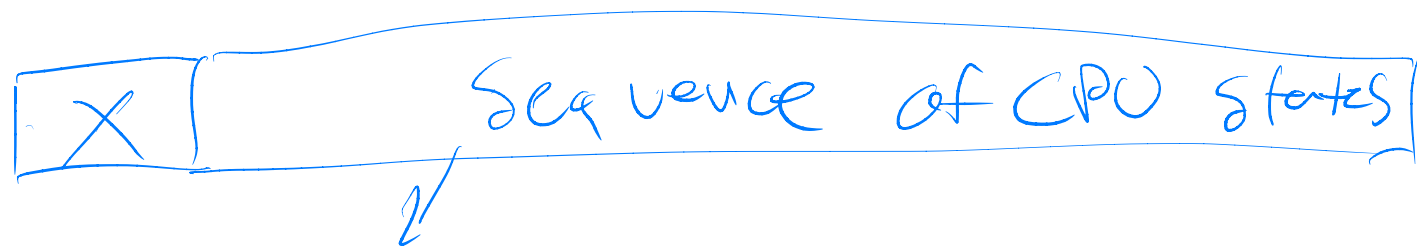
Agenda for this lecture

- Announcements
- Proving memory accesses in CPU-model frontends
- Merkle trees
- memory fingerprinting
- Lookup arguments: doing better for read-only

Reviewing the problem

$P \xrightarrow{\text{compile}} \underline{P'}$

To prove $P'(x)$, Produce transcript t



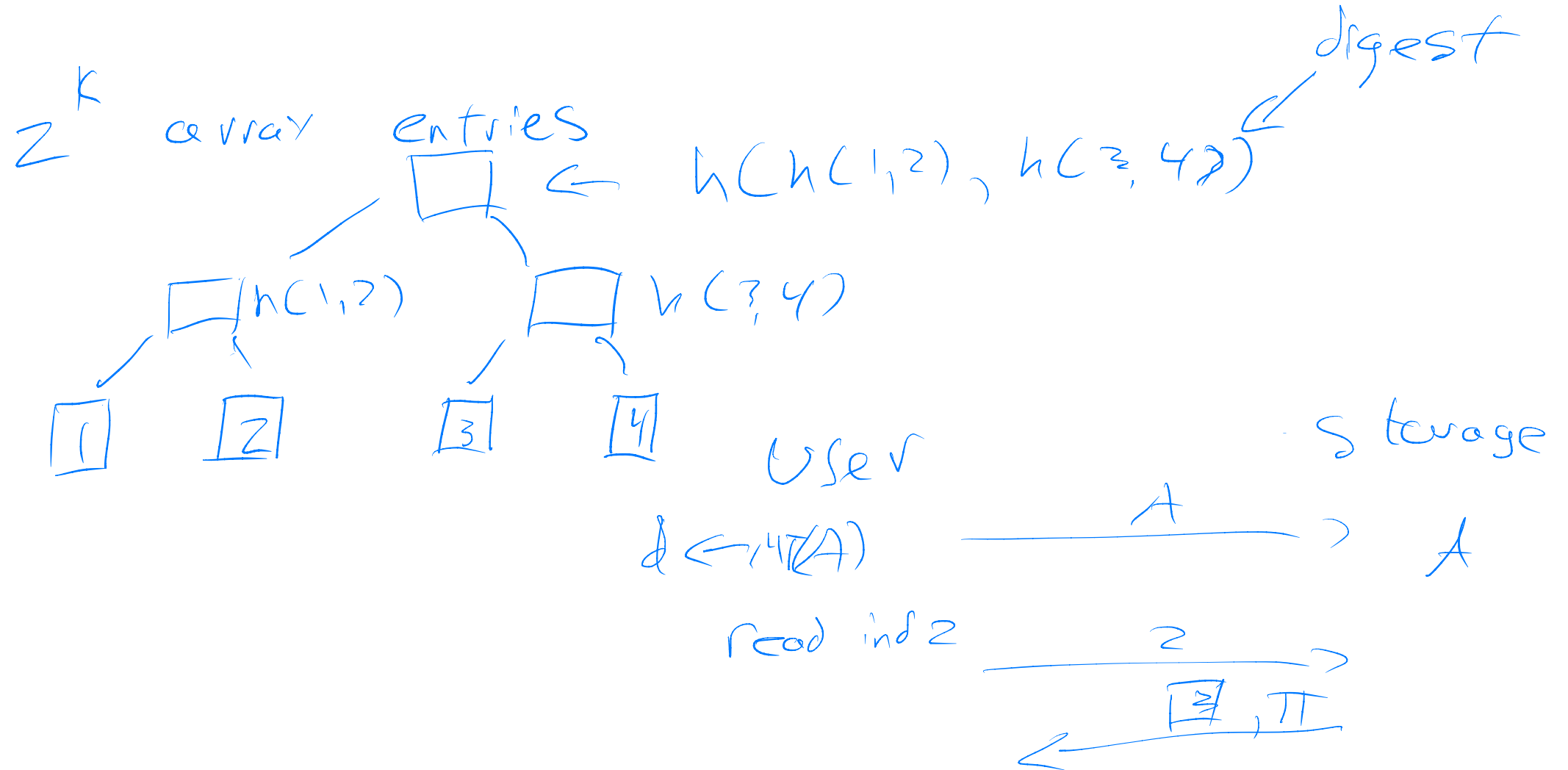
(timestamp, code, list of registers + flags + PC,
read/write information)
 { read add r val
 write add r val

Agenda for this lecture

- Announcements
- Proving memory accesses in CPU-model frontends
- **Merkle trees**
- memory fingerprinting
- Lookup arguments: doing better for read-only

Proving memory via Merkle trees

$$h : \{0, 1\}^L \times \{0, 1\}^L \rightarrow \{0, 1\}^L$$



Proving memory via Merkle trees

$\text{write}(d, A, \text{ind}, \text{val})$:

replace $A[\text{ind}]$ with val ,

give client new hashes

along w/ path

How to prove new consistency

Question:

using MT?

(timestamp, cipcode, regs, mem info)

A of size 2^K , π KL

SHA-256 $\approx 30K$

$$64 \cdot 256 \\ 2^6 \cdot 2^8 = 2^{14}$$

SNARK-friendly: small R/Cs (Poseidon) = 250

Proving memory via Merkle trees

$\text{Digest}(A) \rightarrow d$

$\text{VRead}(d, A, \text{ind}) \rightarrow \text{val}, \pi$

$\text{VWrite}(d, A, \text{ind}, \text{val}) \rightarrow d', A', \pi$

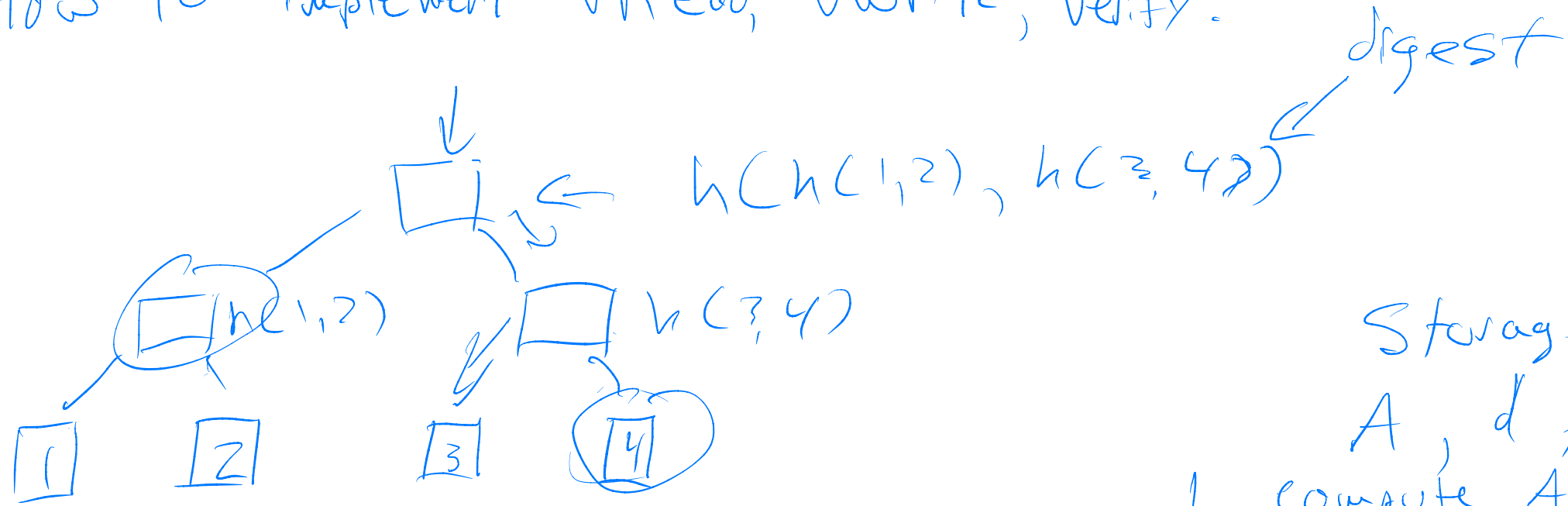
$\text{Verify}(d, \text{ind}, \text{val}, \pi) \rightarrow 0/1$

Soundness (informal): For any program A , the prob
 A outputs array and op sequence that verifies but
is inconsistent is negl.

Claim: If h is collision-resistant,
MT[h] is sound

Proving memory via Merkle trees

How to implement VRead, Vwrite, verify?



Storage

A, d, ind

1. compute $A[ind] = val$
2. compute "copath"
other children
on path from root
to ind
3. $val, copath$
hashes

Agenda for this lecture

- Announcements
- Proving memory accesses in CPU-model frontends
- Merkle trees
- **memory fingerprinting**
- Lookup arguments: doing better for read-only

Memory fingerprinting

MA

Merlin-Arthur

Randomized circuit?

$C(X, w; r)$

↑
chosen by verifier

Soundness over choice of r

How to let verifier choose r ?

P

commit to "pre" witness

C_p

C_p

Δ

$r \leftarrow \mathcal{R}$

r

compute rest
of w , does ZKP
 $C(X, w; r)$

π

$\text{Verify}(r, \pi)$

Memory fingerprinting

- Reduce memory consistency multiset equality checking
1. Add "init write" to every location
 2. For every write in P' , add dummy read right before it (to same location)
 3. For every read, add "write back"
 4. For every write, insert current timestamp into written value
 5. Add final reads of all locs
- Add to checker: if read returns timestamp $>$ current TS, abort

Agenda for this lecture

- Announcements
- Proving memory accesses in CPU-model frontends
- Merkle trees
- memory fingerprinting
- Lookup arguments: doing better for read-only

Lookup arguments